

COMPLETE TECHNICAL REFERENCE

Proxmox VE 5-Node Cluster Setup Command Reference

Step-by-step commands for building a production-ready Proxmox cluster with Ceph, NFS, ZFS RAID-Z2, and LVM storage — designed for future reference.

5	4	300+
PROXMOX NODES	STORAGE TYPES	COMMANDS

PVE 8.x
PROXMOX VERSION

Innspark Infrastructure Team · September 2026 · For Internal Use

📋 Table of Contents

FOUNDATION

01 Network & IP Planning

02 Proxmox Installation Prerequisites

03 Cluster Creation — pve1 (Master Node)

04 Join Nodes 2–5 to Cluster

05 Cluster Verification

STORAGE SETUP

06 Ceph Distributed Storage

07 NFS Storage

08 ZFS RAID-Z2 Storage

09 LVM Storage

MANAGEMENT & HARDENING

10 HA (High Availability) Setup

11 Firewall Configuration

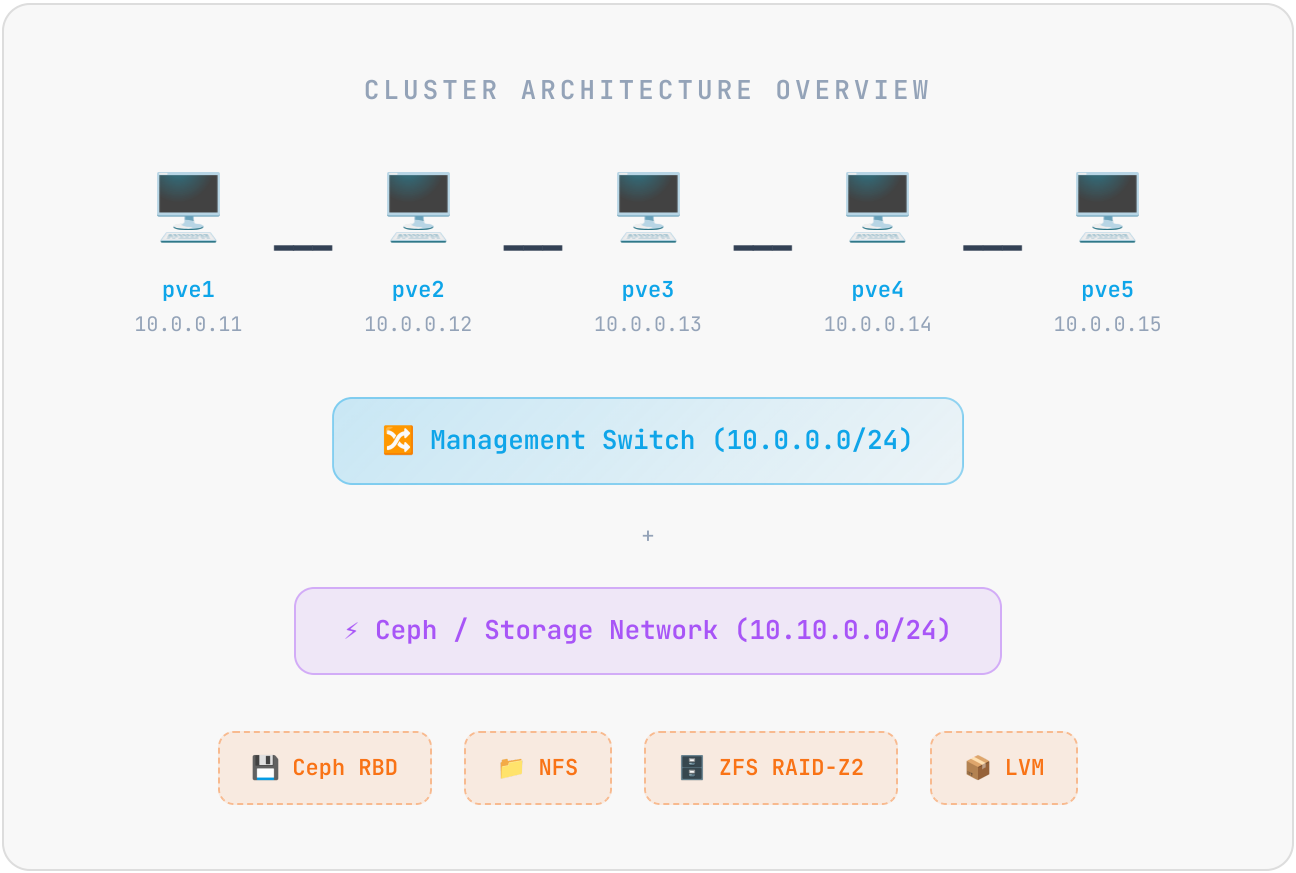
12 Backup & Restore

13 Monitoring & Useful Commands

📡 SECTION 01

Network & IP Planning

Define the IP layout before touching any node. A clean network plan prevents split-brain, corosync timeouts, and storage performance issues.



Node IP Assignment

HOSTNAME	MANAGEMENT IP	CEPH/STORAGE IP	GATEWAY	ROLE
pve1	`10.0.0.11/24`	`10.10.0.11/24`	10.0.0.1	Master / Quorum
pve2	`10.0.0.12/24`	`10.10.0.12/24`	10.0.0.1	Worker
pve3	`10.0.0.13/24`	`10.10.0.13/24`	10.0.0.1	Worker
pve4	`10.0.0.14/24`	`10.10.0.14/24`	10.0.0.1	Worker
pve5	`10.0.0.15/24`	`10.10.0.15/24`	10.0.0.1	Worker
NFS Server	`10.0.0.20/24`	`10.10.0.20/24`	10.0.0.1	NFS Server

Required Firewall Ports

PORT	PROTOCOL	PURPOSE
8006	TCP	Proxmox Web UI (HTTPS)
5900–5999	TCP	VNC Console (SPICE/noVNC)
3128	TCP	SPICE Proxy
5405–5412	UDP	Corosync Cluster Communication
111, 2049	TCP/UDP	NFS RPC / NFS Data
6789	TCP	Ceph MON
6800–7300	TCP	Ceph OSD / MDS
22	TCP	SSH

⚙️ SECTION 02

Proxmox Installation Prerequisites

Run these commands on **every node (pve1–pve5)** right after a fresh Proxmox VE 8.x ISO install. Adjust IPs for each node.

⚠️ **Run on ALL 5 nodes.** Where you see `NODE_IP`, `NODE_NAME`, replace with the correct values per node (see IP table above).

Step 1 — Set Hostname

bash — ALL NODES — pve1..pve5

```
# Replace pve1 with the actual hostname of each node
hostnamectl set-hostname pve1

# Verify
hostname
```

Step 2 — Configure /etc/hosts

bash — ALL NODES

pve1..pve5

```
# Edit hosts file (same content on every node)
cat > /etc/hosts <<'EOF'
127.0.0.1       localhost
# Proxmox Cluster Nodes
10.0.0.11       pve1.innspark.local   pve1
10.0.0.12       pve2.innspark.local   pve2
10.0.0.13       pve3.innspark.local   pve3
10.0.0.14       pve4.innspark.local   pve4
10.0.0.15       pve5.innspark.local   pve5
# NFS Server
10.0.0.20       nfs-server.innspark.local  nfs-server
EOF

# Verify resolution
ping -c2 pve2
```

Step 3 — Configure Network Interfaces

/etc/network/interfaces — pve1 example

pve1

```
# Edit /etc/network/interfaces
cat > /etc/network/interfaces <<'EOF'
auto lo
iface lo inet loopback

# Management Network (eno1 = your physical NIC)
auto eno1
iface eno1 inet manual

auto vmbr0
iface vmbr0 inet static
    address 10.0.0.11/24
    gateway 10.0.0.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
    dns-nameservers 8.8.8.8 8.8.4.4

# Ceph / Storage Network (eno2 = second NIC)
auto eno2
iface eno2 inet static
    address 10.10.0.11/24
EOF

# Apply network config
systemctl restart networking
```

ℹ️ Replace `eno1` / `eno2` with your actual NIC names (use `ip link show` to find them). Adjust the IP addresses for each node (10.0.0.12 for pve2, etc.).

Step 4 — Disable Enterprise Repo & Enable Free Repo

bash — ALL NODES

pve1..pve5

```
# Disable enterprise subscription repo
echo "deb [arch=amd64] http://download.proxmox.com/debian/pve bookworm pve-no-subsc
  > /etc/apt/sources.list.d/pve-no-sub.list

# Comment out enterprise repo
sed -i 's/^deb/#deb/' /etc/apt/sources.list.d/pve-enterprise.list

# Also disable ceph enterprise repo if present
[ -f /etc/apt/sources.list.d/ceph.list ] && \
  sed -i 's/^deb/#deb/' /etc/apt/sources.list.d/ceph.list

# Add no-subscription ceph repo
echo "deb http://download.proxmox.com/debian/ceph-reef bookworm no-subscription" \
  > /etc/apt/sources.list.d/ceph-no-sub.list

# Update and upgrade
apt update && apt full-upgrade -y

# Install useful utilities
apt install -y htop iotop nmap curl wget vim net-tools lsof \
               nfs-kernel-server nfs-common smartmontools
```

Step 5 — Setup SSH Key Authentication Between Nodes

bash — Run on pve1 ONLY

pve1

```
# Generate SSH key on master node
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519

# Copy key to all other nodes
for NODE in pve2 pve3 pve4 pve5; do
  ssh-copy-id -i ~/.ssh/id_ed25519.pub root@${NODE}
  echo "✅ Key copied to ${NODE}"
done

# Test passwordless SSH
for NODE in pve2 pve3 pve4 pve5; do
  ssh root@${NODE} "hostname && uptime"
done
```

🔵 SECTION 03

Create the Cluster — pve1 (Master)

Initialise the Proxmox cluster from the first node. This creates the Corosync ring, sets quorum, and generates join tokens for the other nodes.

🚨 **Run ONLY on pve1.** If you run this on pve2–5, you'll create a separate cluster.

Create the Cluster

bash — pve1 ONLY — pve1

```
# Create cluster named "innspark-cluster" with corosync on management network
pvecm create innspark-cluster \
  --link0 address=10.0.0.11 \
  --link1 address=10.10.0.11

# Verify cluster was created
pvecm status

# List cluster nodes (should show pve1)
pvecm nodes
```

Expected Output

Expected Output

```
Cluster information
-------------------
Name:             innspark-cluster
Config Version:   1
Transport:        knet
Secure auth:      on

Quorum information
------------------
Date:             Fri Sep 19 09:00:00 2026
Quorum provider:  corosync_votequorum
Nodes:            1
Node ID:          0x00000001
Ring ID:          1.5
Quorate:          Yes

Votequorum information
----------------------
Expected votes:   1
Highest expected: 1
Total votes:      1
Quorum:           1
Flags:            Quorate
```

Check Corosync Config

bash pve1

```
# View generated corosync config
cat /etc/pve/corosync.conf

# Check corosync service status
systemctl status corosync
systemctl status pve-cluster
```

🔗 SECTION 04

Join Nodes 2–5 to the Cluster

Each worker node joins the cluster by connecting to the master node (pve1). Run the join command on each node individually.

⚠️ **Join one node at a time.** Wait for each node to successfully join before adding the next. Adding nodes too fast can confuse Corosync quorum.

Join pve2

bash — Run on pve2 — pve2

```
# Join the cluster (you will be prompted for pve1's root password)
pvecm add 10.0.0.11 \
  --link0 address=10.0.0.12 \
  --link1 address=10.10.0.12

# Verify pve2 is in the cluster
pvecm status
pvecm nodes
```

Join pve3

bash — Run on pve3 — pve3

```
pvecm add 10.0.0.11 \
  --link0 address=10.0.0.13 \
  --link1 address=10.10.0.13

pvecm status
```

Join pve4

bash — Run on pve4 pve4

```
pvecm add 10.0.0.11 \
  --link0 address=10.0.0.14 \
  --link1 address=10.10.0.14

pvecm status
```

Join pve5

bash — Run on pve5 pve5

```
pvecm add 10.0.0.11 \
  --link0 address=10.0.0.15 \
  --link1 address=10.10.0.15

pvecm status
```

✅ SECTION 05

Cluster Verification

Run these commands from any node to confirm the cluster is healthy before configuring storage.

bash — Any Node

pve1..pve5

```
# Full cluster status
pvecm status

# List all nodes with IDs
pvecm nodes

# Check quorum (must say Quorate: Yes)
pvecm quorum

# Check corosync ring status
corosync-cfgtool -s

# Check cluster filesystem (pmxcfs)
systemctl status pve-cluster

# View all VMs/containers across cluster
pvesh get /cluster/resources

# Check corosync ring errors
corosync-cfgtool -R

# Verify all 5 nodes visible in UI API
pvesh get /nodes
```

✅ **Success Criteria:** All 5 nodes show in `pvecm nodes`, `Quorate: Yes`, and Expected votes = 5. Now you can configure storage.

💾 SECTION 06

Ceph Distributed Storage

Ceph provides distributed, fault-tolerant block storage (RBD). With 5 nodes, you get replication factor 3 and can tolerate 2 node failures.

🔮 **Ceph Network:** Use the dedicated storage NIC (10.10.0.x) for Ceph traffic. Never mix Ceph OSD traffic with management/VM traffic.

Step 1 — Install Ceph on ALL Nodes

bash — ALL NODES — pve1..pve5

```
# Install Ceph via Proxmox helper (do on each node)
pveceph install --version reef

# OR use apt directly
apt install -y ceph ceph-osd ceph-mon ceph-mgr ceph-mds

# Verify ceph version
ceph --version
```

Step 2 — Initialise Ceph (pve1 ONLY)

bash — pve1 ONLY — pve1

```
# Initialise Ceph cluster with storage network
pveceph init --network 10.10.0.0/24

# Check generated ceph.conf
cat /etc/ceph/ceph.conf
```

Step 3 — Create Monitors (MON) on All Nodes

bash — Run separately on each node

```
# On pve1
pveceph mon create

# On pve2
pveceph mon create

# On pve3
pveceph mon create

# On pve4
pveceph mon create

# On pve5
pveceph mon create

# Verify monitors (from any node)
ceph mon stat
ceph mon dump
```

Step 4 — Create Managers (MGR)

bash

```
# Create MGR on pve1, pve2, pve3 (min 3 for HA)
# On pve1:
pveceph mgr create

# On pve2:
pveceph mgr create

# On pve3:
pveceph mgr create

# Check MGR status
ceph mgr stat
```

Step 5 — Create OSDs (One per Disk per Node)

bash — Run on each node for each data disk

pve1..pve5

```
# List available disks (use disks that are NOT the OS disk)
lsblk -d -o NAME,SIZE,TYPE,MOUNTPOINT
fdisk -l | grep "^Disk /dev"

# Create OSD from /dev/sdb on pve1 (repeat for sdc, sdd etc.)
pveceph osd create /dev/sdb

# Example: Each node has 3 x OSD disks (sdb, sdc, sdd)
pveceph osd create /dev/sdb
pveceph osd create /dev/sdc
pveceph osd create /dev/sdd

# Advanced: Use separate DB/WAL device (NVMe for metadata)
pveceph osd create /dev/sdb \
  --db-dev /dev/nvme0n1 \
  --db-dev-size 60

# Verify OSDs are up
ceph osd stat
ceph osd tree
```

Step 6 — Create Ceph Pool

bash — pve1 pve1

```
# Create a replicated pool for VMs (size=3 means 3 copies)
pveceph pool create vm-pool \
  --size 3 \
  --min-size 2 \
  --pg-autoscale-mode on \
  --add-storages 1

# Verify pool created
ceph osd pool ls
ceph osd pool ls detail

# Check pool stats
rados df

# Full Ceph health check
ceph health detail
ceph status
ceph df
```

Step 7 — Add Ceph Storage to Proxmox GUI

bash — pve1 pve1

```
# Add Ceph RBD storage to Proxmox
pvesm add rbd ceph-storage \
  --pool vm-pool \
  --username admin \
  --krbd 0 \
  --content images,rootdir

# Verify storage added
pvesm status
pvesm list ceph-storage
```

Ceph Health & Monitoring Commands

bash — Ceph Monitoring

```
# Overall status
ceph status
ceph health

# Watch status live
watch -n2 ceph status

# OSD performance
ceph osd perf

# IO stats
ceph iostat

# Disk usage
ceph df detail

# Show PG (placement group) status
ceph pg stat
ceph pg dump_stuck

# Check scrub status
ceph pg dump | grep scrub

# Enable dashboard module
ceph mgr module enable dashboard
ceph dashboard create-self-signed-cert
ceph dashboard set-login-credentials admin <<< "YourPassword"
ceph mgr services
```

📁 SECTION 07

NFS Storage Setup

NFS provides shared storage for ISOs, backups, and VM disk images. Set up an NFS server (on a dedicated node or external server) and mount it on all Proxmox nodes.

Part A — Configure NFS Server (on nfs-server 10.0.0.20)

bash — NFS SERVER (10.0.0.20) nfs-server

```
# Install NFS server
apt install -y nfs-kernel-server nfs-common

# Create export directories
mkdir -p /mnt/nfs/isos
mkdir -p /mnt/nfs/backups
mkdir -p /mnt/nfs/vm-images
mkdir -p /mnt/nfs/snippets

# Set permissions
chmod 777 /mnt/nfs/isos /mnt/nfs/backups \
          /mnt/nfs/vm-images /mnt/nfs/snippets

# Configure exports — /etc/exports
cat > /etc/exports <<'EOF'
/mnt/nfs/isos       10.0.0.0/24(rw,sync,no_subtree_check,no_root_squash)
/mnt/nfs/backups    10.0.0.0/24(rw,sync,no_subtree_check,no_root_squash)
/mnt/nfs/vm-images  10.0.0.0/24(rw,sync,no_subtree_check,no_root_squash)
/mnt/nfs/snippets   10.0.0.0/24(rw,sync,no_subtree_check,no_root_squash)
EOF

# Apply exports
exportfs -ra
exportfs -v

# Enable & start NFS services
systemctl enable nfs-kernel-server rpcbind
systemctl restart nfs-kernel-server rpcbind

# Verify exports
showmount -e localhost
```

Part B — Add NFS to Proxmox (from pve1)

bash — pve1 (Web UI or CLI) pve1

```
# Test NFS mount from pve1 first
showmount -e 10.0.0.20
mount -t nfs 10.0.0.20:/mnt/nfs/isos /mnt/test
ls /mnt/test
umount /mnt/test

# Add NFS ISO storage to Proxmox cluster
pvesm add nfs nfs-isos \
  --server 10.0.0.20 \
  --export /mnt/nfs/isos \
  --content iso \
  --options vers=4

# Add NFS Backup storage
pvesm add nfs nfs-backups \
  --server 10.0.0.20 \
  --export /mnt/nfs/backups \
  --content backup \
  --options vers=4

# Add NFS VM Image storage
pvesm add nfs nfs-vm-images \
  --server 10.0.0.20 \
  --export /mnt/nfs/vm-images \
  --content images \
  --options vers=4

# Add NFS Snippets storage
pvesm add nfs nfs-snippets \
  --server 10.0.0.20 \
  --export /mnt/nfs/snippets \
  --content snippets \
  --options vers=4

# Verify all NFS storages
pvesm status
pvesm list nfs-isos
```

NFS Performance Tuning

bash — NFS SERVER

nfs-server

```
# Increase NFS threads for better performance
echo 'RPCNFSDCOUNT=32' >> /etc/default/nfs-kernel-server

# Tune kernel NFS buffer sizes
cat >> /etc/sysctl.conf <<'EOF'
net.core.rmem_max = 134217728
net.core.wmem_max = 134217728
net.core.rmem_default = 134217728
net.core.wmem_default = 134217728
vm.dirty_ratio = 10
vm.dirty_background_ratio = 5
EOF
sysctl -p

# Restart NFS
systemctl restart nfs-kernel-server
```

🗄 SECTION 08

ZFS RAID-Z2 Storage

ZFS RAID-Z2 provides RAID-6 equivalent protection — it can survive 2 simultaneous disk failures. Ideal for local storage with high data integrity requirements. Requires minimum 6 disks per node for a RAID-Z2 vdev.

ℹ️ **RAID-Z2 Minimum:** 6 disks = 4 data + 2 parity. Recommended: 8 disks (6+2). ZFS is configured per-node (local storage), not shared like Ceph.

Step 1 — Install ZFS Utilities

bash — ALL NODES (or just the ZFS nodes) pve1..pve5

```
# ZFS is built into Proxmox kernel, but install utils
apt install -y zfsutils-linux

# Load ZFS kernel module
modprobe zfs

# Make ZFS load on boot
echo 'zfs' >> /etc/modules

# Verify ZFS version
zfs --version
zpool --version
```

Step 2 — Identify Disks for ZFS Pool

bash — Each Node

```
# List all disks and sizes
lsblk -d -o NAME,SIZE,MODEL,SERIAL,TYPE

# Get disk IDs (use these instead of /dev/sdX for stability)
ls -la /dev/disk/by-id/ | grep -v part

# Check disk health before using
smartctl -a /dev/sdb
smartctl -a /dev/sdc
```

Step 3 — Create ZFS RAID-Z2 Pool

bash — Each Node (adjust disk names) pve1 example

```
# ═══ OPTION A: Using /dev/disk/by-id (RECOMMENDED for production) ═══
zpool create -f zfs-raidz2 \
  raidz2 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0123456 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0234567 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0345678 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0456789 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0567890 \
  /dev/disk/by-id/ata-WDC_WD4TB_WD-WCC4N0678901

# ═══ OPTION B: Using /dev/sdX (simpler but less reliable after reboot) ═══
zpool create -f zfs-raidz2 \
  raidz2 \
  /dev/sdb /dev/sdc /dev/sdd \
  /dev/sde /dev/sdf /dev/sdg

# ═══ OPTION C: 2 RAID-Z2 vdevs for better performance (8 disks) ═══
zpool create -f zfs-raidz2 \
  raidz2 /dev/sdb /dev/sdc /dev/sdd /dev/sde \
  raidz2 /dev/sdf /dev/sdg /dev/sdh /dev/sdi

# Verify pool was created
zpool status zfs-raidz2
zpool list
```

Step 4 — Configure ZFS Pool Settings

bash — ZFS Pool Configuration

```
# Enable LZ4 compression (saves space, barely affects performance)
zfs set compression=lz4 zfs-raidz2

# Set atime=off (performance improvement)
zfs set atime=off zfs-raidz2

# Enable auto-trim for SSDs (skip for HDDs)
zpool set autotrim=on zfs-raidz2

# Set recordsize for VMs/databases (16K or 64K)
zfs set recordsize=64K zfs-raidz2

# Create dedicated dataset for VM images
zfs create zfs-raidz2/vm-data
zfs create zfs-raidz2/backups
zfs create zfs-raidz2/isos

# Set ZFS ARC max RAM (e.g., 8GB = 8589934592 bytes)
echo 'options zfs zfs_arc_max=8589934592' > /etc/modprobe.d/zfs.conf
update-initramfs -u

# View all ZFS properties
zfs get all zfs-raidz2
```

Step 5 — Add ZFS to Proxmox Storage

bash — Each Node (or from pve1 for cluster-wide)

```
# Add ZFS pool as local storage on pve1 (run on each node for local storage)
pvesm add zfspool zfs-raidz2-pve1 \
  --pool zfs-raidz2 \
  --nodes pve1 \
  --content images,rootdir

# For pve2 (run on pve2 or specify from pve1)
pvesm add zfspool zfs-raidz2-pve2 \
  --pool zfs-raidz2 \
  --nodes pve2 \
  --content images,rootdir

# Verify
pvesm status
pvesm list zfs-raidz2-pve1
```

ZFS Maintenance Commands

bash — ZFS Health & Maintenance

```
# Full pool status & health
zpool status -v zfs-raidz2

# Start a scrub (integrity check) — recommended monthly
zpool scrub zfs-raidz2

# Check scrub progress
zpool status zfs-raidz2 | grep -A5 scrub

# List all datasets
zfs list

# ZFS pool I/O stats
zpool iostat zfs-raidz2 2

# Show ARC statistics
arc_summary
cat /proc/spl/kstat/zfs/arcstats | head -20

# Replace a failed disk (sde = old, sdf = new)
zpool replace zfs-raidz2 /dev/sde /dev/sdf
zpool status zfs-raidz2

# Automate monthly scrubs
echo '0 1 1 * * root zpool scrub zfs-raidz2' >> /etc/cron.d/zfs-scrub
```

📦 SECTION 09

LVM Storage

LVM (Logical Volume Manager) provides flexible local block storage. Use LVM-Thin for VM disks (thin provisioning / snapshots) and LVM for raw block devices.

Part A — LVM Setup

bash — Each Node

pve1..pve5

```
# Install LVM2
apt install -y lvm2

# Check available disks (do NOT use the OS disk)
lsblk

# ═══ Step 1: Create Physical Volume (PV) ═══
pvcreate /dev/sdb
pvcreate /dev/sdc

# List PVs
pvdisplay
pvs

# ═══ Step 2: Create Volume Group (VG) ═══
vgcreate vg-pve /dev/sdb /dev/sdc

# List VGs
vgdisplay
vgs

# ═══ Step 3: Create Logical Volumes (LV) ═══
# For VM disks (raw LV — 500GB example)
lvcreate -L 500G -n lv-vmdisk vg-pve

# For all remaining free space
lvcreate -l 100%FREE -n lv-data vg-pve

# List LVs
lvdisplay
lvs

# ═══ Step 4: Format (optional — for data LVs) ═══
mkfs.ext4 /dev/vg-pve/lv-data
mkdir -p /mnt/lvm-data
mount /dev/vg-pve/lv-data /mnt/lvm-data

# Auto-mount on boot
echo '/dev/vg-pve/lv-data  /mnt/lvm-data  ext4  defaults  0  2' >> /etc/fstab
```

Part B — LVM-Thin (Recommended for VMs)

bash — LVM-Thin Pool for VMs

pve1..pve5

```
# Create a Thin Pool (thin provisioning + snapshots)
lvcreate -L 1T -T vg-pve/thin-pool

# Create a thin LV inside the pool
lvcreate -V 200G -T vg-pve/thin-pool -n vm-disk-01

# View thin pool status
lvdisplay vg-pve/thin-pool
lvs -a -o +devices
```

Part C — Add LVM to Proxmox

bash — pve1 (or each node separately) pve1

```
# Add standard LVM VG to Proxmox
pvesm add lvm lvm-storage-pve1 \
  --vgname vg-pve \
  --nodes pve1 \
  --content images,rootdir

# Add LVM-Thin to Proxmox (RECOMMENDED for VMs with snapshots)
pvesm add lvmthin lvmthin-pve1 \
  --vgname vg-pve \
  --thinpool thin-pool \
  --nodes pve1 \
  --content images,rootdir

# For pve2 (run on pve2's VG)
pvesm add lvmthin lvmthin-pve2 \
  --vgname vg-pve \
  --thinpool thin-pool \
  --nodes pve2 \
  --content images,rootdir

# Verify storage
pvesm status
pvesm list lvmthin-pve1
```

LVM Useful Commands

bash — LVM Management

```bash
# Extend a VG by adding a new disk
pvcreate /dev/sdd
vgextend vg-pve /dev/sdd

# Extend an LV by 100GB
lvextend -L +100G /dev/vg-pve/lv-data
resize2fs /dev/vg-pve/lv-data  # for ext4

# Take LVM Snapshot of a VM disk
lvcreate -L 10G -s -n snap-vm100 /dev/vg-pve/vm-100-disk-0

# Remove LVM Snapshot
lvremove /dev/vg-pve/snap-vm100

# Scan all PVs/VGs/LVs
pvscan
vgscan
lvscan

# Show full details
pvdisplay -v
vgdisplay -v
lvdisplay -v
```

🔄 SECTION 10

High Availability (HA) Setup

Proxmox HA automatically restarts VMs/containers on other nodes when a node fails. Requires shared storage (Ceph or NFS).

bash — HA Configuration

pve1

```
# Create HA group with all 5 nodes
ha-manager groupadd ha-group-all \
  --nodes pve1:2,pve2:1,pve3:1,pve4:1,pve5:1 \
  --restricted 0 \
  --nofailback 0

# List HA groups
ha-manager groupconfig

# Add a VM (VMID 100) to HA
ha-manager add vm:100 \
  --group ha-group-all \
  --state started \
  --max-restart 3 \
  --max-relocate 3

# Add a container (CTID 200) to HA
ha-manager add ct:200 \
  --group ha-group-all \
  --state started

# List all HA resources
ha-manager status

# Remove a VM from HA
ha-manager remove vm:100

# Check HA manager service
systemctl status pve-ha-lrm
systemctl status pve-ha-crm
```

Fencing (Watchdog)

bash — Watchdog / Fencing

```
# Check watchdog config
cat /etc/pve/datacenter.cfg

# Enable software watchdog (if no hardware watchdog)
echo 'softdog' >> /etc/modules
modprobe softdog

# Verify watchdog
ls /dev/watchdog*

# Configure in datacenter.cfg
pvesh set /cluster/options \
  --ha '{"shutdown_policy":"freeze"}' \
  --watchdog 'model=i6300esb'
```

🔥 SECTION 11

Firewall Configuration

Proxmox has a built-in firewall. Configure it to secure the cluster while allowing required cluster and storage communications.

bash — Firewall Rules

pve1

```
# Enable datacenter firewall
pvesh set /cluster/firewall/options \
  --enable 1 \
  --policy-in DROP \
  --policy-out ACCEPT

# Create a cluster-level security group for management
pvesh create /cluster/firewall/groups \
  --group cluster-mgmt \
  --comment "Cluster Management Rules"

# Allow SSH from management network
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto tcp --dport 22 \
  --source 10.0.0.0/24 --comment "SSH"

# Allow Proxmox Web UI
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto tcp --dport 8006 \
  --source 10.0.0.0/24 --comment "Proxmox WebUI"

# Allow Corosync cluster traffic
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto udp --dport 5405:5412 \
  --comment "Corosync"

# Allow Ceph MON
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto tcp --dport 6789 \
  --source 10.10.0.0/24 --comment "Ceph MON"

# Allow Ceph OSD
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto tcp --dport 6800:7300 \
  --source 10.10.0.0/24 --comment "Ceph OSD"

# Allow NFS
pvesh create /cluster/firewall/groups/{cluster-mgmt}/rules \
  --action ACCEPT --type in --proto tcp --dport 2049 \
  --source 10.0.0.0/24 --comment "NFS"
```

```
# List all firewall rules
pvesh get /cluster/firewall/rules
```

💽 SECTION 12

Backup & Restore

Proxmox Backup Server (PBS) integration or local vzdump backups.

vzdump — Manual Backup

bash — Backup Commands

```
# Backup VM 100 to NFS backup storage (stops VM)
vzdump 100 --storage nfs-backups --mode stop

# Backup VM 100 with snapshot (no downtime, requires QEMU agent)
vzdump 100 --storage nfs-backups --mode snapshot

# Backup VM 100 with suspend mode
vzdump 100 --storage nfs-backups --mode suspend

# Backup container 200
vzdump 200 --storage nfs-backups --mode snapshot

# Backup ALL VMs on node pve1 to NFS
vzdump --all --storage nfs-backups --node pve1 --mode snapshot

# Backup with compression
vzdump 100 --storage nfs-backups --compress zstd

# Schedule automatic backups (via API)
pvesh create /cluster/backup \
  --enabled 1 \
  --storage nfs-backups \
  --schedule 'sun 02:00' \
  --mode snapshot \
  --compress zstd \
  --prune-backups 'keep-last=7,keep-weekly=4'
```

Restore

bash — Restore Commands

```
# List available backups
pvesh get /nodes/pve1/storage/nfs-backups/content

# Restore VM from backup file
qmrestore /mnt/pve/nfs-backups/dump/vzdump-qemu-100-2026_09_19-02_00_00.vma.zst \
  100 --storage ceph-storage

# Restore container from backup
pct restore 200 /mnt/pve/nfs-backups/dump/vzdump-lxc-200-2026_09_19-02_00_00.tar.zs
  --storage ceph-storage
```

📊 SECTION 13

Monitoring & Useful Daily Commands

Quick reference for day-to-day cluster management.

Cluster Status

bash — Cluster Commands

```
# Full cluster overview
pvecm status

# List all VMs across cluster
pvesh get /cluster/resources --type vm

# List all nodes with resources
pvesh get /nodes

# Node CPU/Memory usage
for NODE in pve1 pve2 pve3 pve4 pve5; do
  echo "═══ $NODE ═══" && ssh root@$NODE "free -h && top -bn1 | head -5"
done

# All storage status
pvesm status

# Disk usage across all storages
pvesm list --storage ceph-storage
pvesm list --storage nfs-backups

# Live resource monitor
watch -n3 "pvesh get /cluster/resources"
```

VM & Container Management

bash — VM & Container Commands

```
# List all VMs (QEMU)
qm list

# List all containers (LXC)
pct list

# Start/Stop/Restart VM
qm start 100
qm stop 100
qm reboot 100

# Migrate VM to another node (live migration)
qm migrate 100 pve2 --live

# Migrate with storage (different storage backend)
qm migrate 100 pve2 --live --targetstorage ceph-storage

# Clone VM
qm clone 100 200 --name new-vm --full

# Snapshot VM
qm snapshot 100 snap1 --description "Before update"
qm listsnapshot 100
qm rollback 100 snap1
qm delsnapshot 100 snap1

# Get VM config
qm config 100

# Monitor VM in real time
qm monitor 100
```

Log Locations

LOG	LOCATION
Proxmox cluster log	`/var/log/pve-cluster/`
Corosync log	`/var/log/corosync/corosync.log`
Ceph log	`/var/log/ceph/`
VM/LXC task logs	`/var/log/pve/`
vzdump backup log	`/var/log/vzdump/`
System journal	`journalctl -u pve-cluster`
HA manager log	`journalctl -u pve-ha-crm`

🛠️ SECTION 14

Troubleshooting Cheat-Sheet

Common issues and their fixes.

Cluster Issues

bash — Cluster Troubleshooting

```
# Cluster not quorate (split-brain)
pvecm expected 1   # DANGEROUS! Only use if you know what you're doing

# Restart cluster services
systemctl restart pve-cluster corosync

# Check corosync ring errors
corosync-cfgtool -s

# Force node removal from cluster (dead node)
pvecm delnode pve5

# Re-add a node that was removed
pvecm add 10.0.0.11 --link0 address=10.0.0.15

# Cluster log
journalctl -u corosync --since "1 hour ago"
journalctl -u pve-cluster --since "1 hour ago"
```

Ceph Issues

bash — Ceph Troubleshooting

```
# Check health warnings
ceph health detail

# Restart a specific OSD (OSD 5)
systemctl restart ceph-osd@5

# Mark OSD as out before removing
ceph osd out 5
ceph osd down 5
ceph osd rm 5
ceph osd crush remove osd.5

# Check slow OSD requests
ceph daemon osd.0 dump_ops_in_flight

# Fix clock skew warning
apt install -y chrony
systemctl enable chrony
systemctl restart chrony
timedatectl status

# Full Ceph reset (DANGEROUS — wipes Ceph!)
# pveceph purge
```

ZFS Issues

bash — ZFS Troubleshooting

```
# Pool showing degraded
zpool status -v

# Clear ZFS errors after fixing
zpool clear zfs-raidz2

# Import a pool that wasn't imported
zpool import zfs-raidz2

# Export pool before moving to another node
zpool export zfs-raidz2

# Force import if needed
zpool import -f zfs-raidz2

# Check for ZFS errors in syslog
dmesg | grep -i zfs
journalctl | grep -i zfs
```

NFS Issues

bash — NFS Troubleshooting

```
# NFS mount hanging / stale
umount -f -l /mnt/pve/nfs-backups

# Check NFS server exports
showmount -e 10.0.0.20

# Check NFS mounts on client
mount | grep nfs

# Restart NFS server
systemctl restart nfs-kernel-server

# Check RPC status
rpcinfo -p 10.0.0.20

# Test NFS performance
dd if=/dev/zero of=/mnt/pve/nfs-backups/testfile bs=1M count=1000 oflag=direct
```

LVM Issues

bash — LVM Troubleshooting

```
# LVM not seeing disks
pvscan --cache
vgscan
lvscan

# Fix LVM after disk replacement
vgchange -ay vg-pve

# Thin pool overprovisioned
lvs -a -o +devices
# Resize thin pool
lvextend -L +500G vg-pve/thin-pool

# Repair a VG
vgck vg-pve
vgreduce --removemissing vg-pve
```

Storage Quick Reference

STORAGE	TYPE	SHARED?	SNAPSHOTS	LIVE MIGRATION	BEST FOR
Ceph RBD	Distributed	✅ Yes	✅ Yes	✅ Yes	VM disks, HA
NFS	Network	✅ Yes	❌ No	✅ Yes	ISOs, Backups
ZFS RAID-Z2	Local	❌ No	✅ Yes	❌ No*	High IOPS, local
LVM-Thin	Local	❌ No	✅ Yes	❌ No*	Speed, local VMs

* Offline migration with storage migration is possible

🎉 **Cluster Setup Complete!** You now have a fully operational 5-node Proxmox cluster with Ceph, NFS, ZFS RAID-Z2, and LVM storage. Access the Web UI at `https://10.0.0.11:8006` to manage the entire cluster.
